



HASHDIT

HashDit

Smart Contract Security

ListaDao CreditBroker Fix

Security Audit Report

ListaDao

2 Sep 2026 – 11 Sep 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	2 Sep 2026 - 11 Sep 2026
Overall Risk	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/238
Commit	bca6a3dcfea903327b025c03511edc06aad26748

Smart Contract List

No.	Contract Name	Verdict	Details
1	ICreditBroker.sol	Green	-
2	CreditBrokerMath.sol	Green	-
3	CreditBrokerOperatorLib.sol	Informational	[I01] [I02] [I03] [I04] [I05] [I06] [I07]
4	MoolahOperateLib.sol	Green	-
5	CreditBroker.sol	Informational	[I06] [I07]

Notes:

编号	问题
H1	loan token 价格下跌 → minLoan (代币数) 升高 → 存量仓搁浅
M	治理上调 minLoanValue → 同样搁浅

借贷场景下,
minLoan 可能会变化。

① $\text{minLoan} = \text{minLoanValue} * 10^{**}\text{loanDecimals} / \text{peek}(\text{loanToken})$ // minLoan 以多少 loanToken 来衡量

JavaScript

```
// Moolah.sol:892
/// @inheritdoc IMoolahBase
function minLoan(MarketParams memory marketParams) public view returns (uint256)
{
    uint256 price = IOracle(marketParams.oracle).peek(marketParams.loanToken);
    uint8 decimals = IERC20Metadata(marketParams.loanToken).decimals();
    return price == 0 ? 0 : minLoanValue.mulDivDown(10 ** decimals, price);
}

/// The minimum loan token position value
uint256 public minLoanValue;
```

所以 minLoan 跟 loanToken 的 price 成反比关系, 跟 minLoanValue 成正比关系。

peek(loanToken) ↓, minLoan ↑

minLoanValue ↑, minLoan ↑

以上 2 种情况会导致 minLoan ↑

② Moolah 借贷协议中, minLoan

在 Moolah 里, position 不是泛指「用户在协议里的所有钱」, 而是:
某个用户 × 某个 market 的一组状态

JavaScript

```
// IMoolah.sol
struct Position {
    uint256 supplyShares; // 用户 张三 存进这个市场 的 loan token (当出借人/流动性提供者)
    uint128 borrowShares; // 用户 张三 从这个市场借出 的 loan token (当借款人)
```

```
uint128 collateral;    // 用户 张三 押在这个市场 的抵押品
}
```

一句话总结：

- position 是 supply + borrow + collateral 三个角色在这个 market 上的快照
- 同一个人可以同时 supply 和 borrow

minLoan 是干什么的？

minLoan 对 supply 侧 和 borrow 侧 分别生效：

- 若 supplyShares > 0 → 供应资产必须 = 0 或 ≥ minLoan
- 若 borrowShares > 0 → 借款资产必须 = 0 或 ≥ minLoan
 - 这是协议故意设的 最小有效借款规模，不是会计错误。目的是：不允许账户长期挂着「还有一点点债务」的状态。
- collateral 不受 minLoan 约束

不是「用户在 Moolah 所有 market 的总存款」，而是 每个 market 内、每一侧（借/存）各自的最小有效规模。

minLoan 设计目的：

1. 防止 dust 头寸
2. 借款/供应/share 换算有 rounding，极小头寸经济上无意义，链上操作成本却很高。Moolah 要求：头寸要么 清零，要么 ≥ minLoan。
3. borrow / repay / supply / withdraw 统一约束

3minLoan变化：

有两种可能position会低于minLoan，也就是把头寸推入 (0, minLoan) 这个禁止中间态。用户position的剩余本金是固定的（比如一直欠 20 lisUSD），但 门槛 minLoan 会动。

两种触发路径都是 minLoan ↑，结合1：

1. peek(loanToken) ↓, minLoan ↑ -> 对应的现实场景：脱钩 //因为loanToken是lisUSD
2. minLoanValue ↑, minLoan ↑ -> 对应的现实场景：治理上调 //minLoanValue 最小贷款金额

以上 2 种情况会导致 minLoan ↑

而如果 minLoan ↓，地板降低，存量头寸通常更安全

举例：

场景 1: lisUSD 脱锚（价格下跌）

	lisUSD 价格	minLoan (代币)	张三的position
--	-----------	---------------	-------------

借款时	\$1.00	15 lisUSD	借 20 lisUSD ✓
脱锚后	\$0.50	30 lisUSD	仍欠 20 lisUSD ✗

minLoanValue最小贷款金额不变, 但 lisUSD 便宜了, 要凑够 \$15 就需要 更多 lisUSD, 所以代币计的 minLoan 反比于价格上升。

场景 2: 治理上调 minLoanValue

	minLoanValue	minLoan(代币)	张三的头寸
借款时	\$15	15 lisUSD	借 20 lisUSD ✓
上调后	\$30	30 lisUSD	仍欠 20 lisUSD ✗

价格不变, 但 minLoanValue最小贷款金额直接翻倍, 同样把 20 推入死区。

所以以minLoanValue和 minLoan的关系是？

minLoanValue以美元计价, 因此, minLoan以贷款代币计价, 其价格随贷款代币价格波动。

④低于minLoan 门槛(死区)的impact?

(0, minLoan) = 禁止区

> 本身不会自动清算或罚息, 但会:

JavaScript

掉进 (0, minLoan)

```

|
|— Moolah
|   borrow/repay 结束后 _checkBorrowAssets
|   → revert "remain borrow too low"
|   → 不能 partial repay 留在死区; 只能清到 0
|

```

//Moolah.sol

```

function _checkBorrowAssets(MarketParams memory marketParams, address account)
internal view returns (bool) {
    Id id = marketParams.id();
    if (position[id][account].borrowShares == 0) {
        return true;
    }

    return
        uint256(position[id][account].borrowShares).toAssetsDown(
            market[id].totalBorrowAssets,
            market[id].totalBorrowShares
        ) >= minLoan(marketParams);
}

```

对剩余本金 R, 合法状态只有: $R=0$ 或 $R \geq \minLoan$

禁止 $0 < R < \text{minLoan}$

CreditBroker 是 Moolah 某一个 market 上的 broker 包装层。用户通常不直接调 Moolah borrow/repay, 而是:

用户

↓ borrow / repay

CreditBroker (broker 层, 记多笔 fixed loan)

↓ 汇总后操作

Moolah 的 position[marketId][user] (只有一份 aggregate 状态)

```
├─ CreditBroker (旧) //信用贷合约
│   └─ _validatePositions 扫全户
//全户
/// @dev how many fixed loan positions a user can have
///仓位存在本合约的 fixedLoanPositions[user]
uint256 public maxFixedLoanPositions;

├─
│   └─ revert "below min loan"
│       └─ 一笔搁浅锁死 borrow/repay/还利息
└─ CreditBroker (新)
    └─ 只校验被 touch 的那笔 + repayAll 清户出口
        └─ 死区仍禁止「部分还搁浅笔」
        └─ 不再禁止「操作其他笔 / 新借合规笔」
```

JavaScript

```
/**
 * @dev Validate that the user's positions meet the minimum loan requirement
 * @param user The address of the user
 */
function _validatePositions(address user) internal view {
    FixedLoanPosition[] memory fixedPositions = fixedLoanPositions[user];
    // assume valid first
    bool isValid = true;
    uint256 minLoan = MOOLAH.minLoan(MOOLAH.idToMarketParams(MARKET_ID));

    // check fixed positions
    for (uint256 i = 0; i < fixedPositions.length; i++) {
        FixedLoanPosition memory _fixedPos = fixedPositions[i];
        uint256 fixedPosDebt = _fixedPos.principal - _fixedPos.principalRepaid;
        if (fixedPosDebt > 0 && fixedPosDebt < minLoan) {
            isValid = false;
        }
    }
}
```

```
require(isValid, "below min loan");
}
```

// 是一个 internal view 函数, 会遍历用户所有 fixed positions, 只要有一个剩余本金落在 (0, minLoan) 就 revert "below min loan"。

JavaScript

_validatePositions - 链上强制校验 (会 revert)
定义位置: CreditBroker.sol (internal view)

直接调用者只有 2 个 internal 函数:

internal 函数 调用时机

- _borrow(amount, termId): 借款完成、状态更新后
- _repay(amount, posId, onBehalf, receivedInterest): 还款完成、状态更新后

Impact

1. 借款侧 (borrow)

已经落在死区里 (比如剩 20, minLoan = 30):

操作	结果
部分还款 (还 5, 剩 15)	❌ revert "remain borrow too low"
部分还款 (还 10, 剩 10)	❌ 同上
一次性还清全部	✅ 可以, borrowShares → 0
新借一笔	取决于 aggregate; CreditBroker 旧版还会被全局校验卡住

想新借一笔也低于地板 (比如借 1 lisUSD, minLoan = 15):

- ❌ 直接被拒 ("below min loan")

2. 供应侧 (supply)

逻辑对称: 若张三还有 supply shares, 对应资产不能落在 (0, minLoan)。

操作	结果
部分 withdraw 到死区	❌ revert "remain supply too low"
首次 supply 低于 minLoan	❌ revert
一次性全部 withdraw	✅ 可以

3. 抵押品 (collateral)

不受 minLoan 影响。死区只约束 borrow/supply, 不约束 collateral 数量。

4. CreditBroker 旧版额外伤害

即使 Moolah aggregate 总借款仍 $\geq$ minLoan (比如 $20 + 40 = 60$), 只要 任意一笔 fixed position 掉进死区, 旧版 `_validatePositions` 会:

-  阻塞还 其他健康头寸
-  阻塞 只还利息
-  阻塞 新借款

5 CreditBroker 的问题:

JavaScript

minLoan 变大 (脱锚 / 上调 minLoanValue)

- 某笔仓落入 $(0, \text{minLoan})$
- 不能部分还
- 旧版全局校验 → 整户操作全废①
- 又没有 `repayAll`, 全额还还要猜金额② //被冻结的仓位 (本金+累计利息+15%罚息)
- 坏账仓也参与校验, 进一步误伤③
- 用户感觉: 账户冻住, 只能硬凑一次还清

6 PR修复

PR #238 要修的是 CreditBroker 账户被 $(0, \text{minLoan})$ 死区“锁死”的问题: 不是 minLoan 机制本身错了, 而是 CreditBroker 在 Moolah 约束之上又加了一层错误的全局校验, 把本来只影响单个头寸的约束放大成了整户冻结。

CreditBroker 的 fixed position 是 broker 在 Moolah 之上的子账本, 一笔笔 fixed loan; PR 修的是这层子账本的全局校验 bug。

一句话总结

核心问题: 当 `lisUSD` 脱锚或 `minLoanValue` 上调时, 原本合法的借款头寸会落入 $(0, \text{minLoan})$ 区间; Moolah 本来就不允许部分还款留在这个区间, 但旧版 CreditBroker 还会 遍历账户内所有头寸, 只要有一个“搁浅”, 就 revert 掉该用户的一切操作 (还其他头寸、只还利息、新借款), 且 没有可靠的“一次性清户”出口。

旧 CreditBroker: 连还别的头寸、借新钱也 

PR #238 后: 还别的头寸 , 还这笔必须 `repayAll` / 全额还清

7 Tests — 1064 passing, 20 new

- Reproduces the bug: a 20-token position opened at lisUSD \$1.00 (floor 15) becomes unrepayable at \$0.50 (floor 30) — partial repay reverts below min loan — and repayAll clears it. Same for a minLoanValue raise.
- Isolation: a stranded position no longer blocks repay or borrow of others, while a new below-floor borrow is still rejected.
- Equivalence: repayAll charges exactly what full per-position repay charges, cross-checked via vm.snapshotState across accrue-interest, upfront-interest, and post-grace penalized regimes; and the pulled amount equals the previewRepayAll quote.
- Bad-debt-only accounts, insufficient funds (reverts with no partial state change), pause, third-party payer, post-repayAll withdraw/borrow unblocking, and gas at maxFixedLoanPositions (1.01M gas at 100 positions — no DoS on the only available exit).

⑧ 注意点

部署

仅代码操作：这本身并不能(0, minLoan)挽救已搁置的主网仓位。新实现必须通过时间锁进行部署和升级。建议在安排调度之前检查当前是否有任何活跃仓位位于该区间内。

升级前先摸清有多少人卡在死区。

Findings

[I01] repayAll concentrates broker revenue into a single vault supply and widens the share price front running window

Contract CreditBrokerOperatorLib.sol

Severity Informational

Description

CreditBrokerOperatorLib.repayAll() sums a user's entire outstanding interest, delay penalty, and all bad debt principal into one transfer and settles it through the relay in a single supplyToVault() call. That call supplies bare assets into the market on the vault's behalf without minting any new vault shares, so the vault's totalAssets() and therefore its share price step upward in the block the flush lands.

JavaScript

```
FixedLoanPosition[] memory positions = fixedLoanPositions[onBehalf];
(uint256 interestAndPenalty, uint256 badDebtPrincipal, uint256
debtAtMoolah, uint256 totalDebt) = CreditBrokerMath
.previewRepayAllAmounts(onBehalf, positions, ctx.graceConfig,
address(ctx.moolah), ctx.marketId);
require(totalDebt > 0, "nothing to repay");
```

Description

The repayment notification is predictable, given the relay only forwards accumulated funds to the vault once its running balance reaches $\geq \text{minLoan}$. A user who sees the balance approach that threshold, or sees the triggering repay in the mempool, can sandwich the step, by depositing into the vault immediately before the flush to mint shares at the lower price, let the notification raise the price, then withdraw immediately after to redeem at the higher price.

Impact

This front running vector already existed for the per position repay() and repayInterestWithLista() paths, but only relates to singular, separated positions. repayAll() aggregates a whole account's interest, penalty, and bad debt principal into one much larger flush, producing a correspondingly larger single step for an attacker to target.

The existing mitigation, BrokerInterestLockBuffer, neutralizes this regardless of flush size, because it locks each notified amount and releases it linearly over its configured duration, so no moreover, the vault only subtracts the locked amount from totalAssets() when a lock buffer is configured, that is when $\text{buf} \neq \text{address}(0)$. On a credit vault with no buffer set, only the minLoan batching bounds the step, and that is exactly the configuration where a large repayAll() flush creates the biggest instantly visible price jump and the largest extractable sandwich.

Recommendation	Require a non zero BrokerInterestLockBuffer on every vault that backs a credit market, and treat that buffer as a mandatory deployment prerequisite before repayAll() is enabled. As defense in depth, consider asserting that the target vault has a lock buffer set on the settlement path, or capping the amount supplied per flush so that a single repayAll() cannot create an outsized step, while retaining the minLoan batching already in place.
Status	Acknowledged, it is presumed that there will be a non-zero BrokerInterestLockBuffer on every vault that backs a credit market to limit the front-running attack vector.

[I02] Possible additional CreditBroker actions and duplicated debt logic migration

Contract	CreditBrokerOperatorLib.sol
Severity	Informational
Description	Description After the repay family was relocated into CreditBrokerOperatorLib, CreditBroker compiles to 21,659 bytes against the 24,576 byte EIP-170 runtime limit, leaving roughly 2.9 KB of headroom. Several state changing actions still sit in the contract body and could follow the same DELEGATECALL pattern to restore headroom if future changes grow the contract again. The strongest candidates are the following • Borrow path together with its term lookup and position creation, This is the largest market action still in the contract with position struct construction, _getTermById loop, the Moolah borrow with balance assertion, the min-loan check, and the event. The following caveats should be taken into account: ○ It reads fixedTerms (pass as a storage array ref, like fixedLoanPositions) and _getTermById should move with it. ○ It increments fixedPosUuid. Value-type state variables can't be passed by storage reference, only mappings/arrays/structs can. So return the new uuid and let the broker store it (fixedPosUuid = CreditBrokerOperatorLib.borrow(...)), or fold the counter into a storage struct. ○ The userNotPenalized and noDebt modifiers can't live in a library. Keep them on the broker's external wrapper (or inline their checks in the lib body); the guard modifiers (whenNotPaused, nonReentrant, whenBorrowNotPaused, syncCreditScore, marketIdSet) stay on the wrapper regardless. • The collateral supply and withdraw paths together with the debt burn helper. These mirror the repay migration almost exactly, where they touch the

fixedLoanPositions mapping (the bad-debt loop in _withdrawCollateral), COLLATERAL_TOKEN, and MOOLAH.

- There are also two view side duplications, getUserTotalDebt reimplements the exact accumulation loop already present in CreditBrokerMath.getTotalDebt, and the penalized position checks repeat the same loop over a user's positions in more than one place.

Impact

Other than contract size issues, the duplication carries a real maintainability risk that is independent of contract size. If a future change towards the CreditBroker market updates the debt accumulation in one copy but not the other, the value reported by the view and the value used by the internal accounting will disagree, and any integrator or off chain system that trusts the view will read a figure that no longer matches the math the contract actually applies.

Any migration must respect the constraints that was already performed for the repay migration. Value type state such as the position id counter cannot be passed to a library by storage reference, so it has to be returned by the library function and written back by the broker. Guard modifiers and role checks must stay on the external wrappers, since a library cannot carry modifiers. Every event re-emitted from the library must keep a signature identical to ICreditBroker, otherwise the emitted topic will change and off chain indexers will stop matching those events.

Recommendation

Treat the remaining 2.9 KB as a budget rather than free space. If new logic is added for the CreditBroker markets, relocate the collateral and liquidation actions first, since they only need the positions mapping and the operator context, and move the borrow path afterward, since it additionally needs the term array passed by storage reference and the incremented position id returned to the broker

Status

Acknowledged, the suggestions can be considered in future upgrades if applicable.

[I03] Partial repay paths omit the pre accrual that protects repayAll

Contract

CreditBrokerOperatorLib.sol

Severity

Informational

Description

Description

repayAll calls Moolah's accrueInterest before it prices the debt, so the asset figure it precomputes matches what Moolah charges when it repays by shares, and its assetsRepaid <= maxAmount guard holds regardless of the market interest rate.

JavaScript

```
function repayAll(
```

```

mapping(address => FixedLoanPosition[]) storage fixedLoanPositions,
OperatorContext memory ctx,
address onBehalf
) external {
    require(onBehalf != address(0), "zero address");
    address user = msg.sender;

    // accrue first so `debtAtMoolah` below is exact for this block
    MarketParams memory marketParams =
    ctx.moolah.idToMarketParams(ctx.marketId);
    ctx.moolah accrueInterest(marketParams);

```

repay and repayInterestWithLista do not perform this pre accrual. They derive the Moolah repayment from the user supplied amount by converting it to shares with toSharesDown on the market totals read at that moment, and they approve only amount to Moolah. Moolah then accrues interest internally as the first step of its own repay call, which raises totalBorrowAssets while the borrow shares stay unchanged.

On a market whose interest rate model is not zero rate, this ordering means the shares computed from the stale totals can cost more assets after Moolah accrues than the amount the broker approved and transferred. Moolah's transfer then exceeds the approval and the whole repayment reverts.

Impact

The impact is that ordinary repayment, both repay and interest repayment with LISTA, can become unexecutable on a non zero rate market, even though repayAll still works because it pre accrues. Every one of these paths silently depends on the broker market running a zero rate interest model.

That assumption is stated in code comments but is not enforced anywhere, since neither market creation nor setMarketId checks the interest rate model. There is no fund loss.

Recommendation

For defense in depth and for consistency with repayAll, additionally have repay and repayInterestWithLista accrue Moolah interest before deriving shares, or size the Moolah approval to the post accrual asset cost, so that a stale read can never produce an allowance shortfall revert. If not, consider not having any CreditBroker markets with a non-zero IRM.

Status

Fixed, accrual is moved into MoolahOperateLib.repayToMoolah, before the market totals are read, which covers both repay() and repayInterestWithLista() (the latter routes through the former).

Note that this is defense in-depth because a direct Moolah accrual is unnecessary considering we always expect a zero-rate IRM in the current design of CreditBroker markets.

[I04] repayAll omits the per position RepaidFixedLoanPosition event

Contract CreditBrokerOperatorLib.sol

Severity **Informational**

Description

The repay path emits a full RepaidFixedLoanPosition event for each repayment, carrying the per position breakdown of principal, interest, penalty, and the isBadDebt flag.

JavaScript

```
// emit event
emit RepaidFixedLoanPosition(
  onBehalf,
  posId,
  position.principal,
  position.start,
  position.end,
  position.apr,
  position.principalRepaid,
  principalRepaid,
  repayInterestAmt,
  penalty,
  position.interestRepaid,
  position.isBadDebt
);
```

Description

The repayAll path does not. When it clears a user's positions it emits only a FixedLoanPositionRemoved event per posId plus a single aggregate AllPositionsRepaid(onBehalf, totalDebt), and it never emits RepaidFixedLoanPosition for the positions it closes.

JavaScript

```
// emit per-position removal events, then wipe the array
for (uint256 i = 0; i < positions.length; i++) {
  emit FixedLoanPositionRemoved(onBehalf, positions[i].posId);
}
delete fixedLoanPositions[onBehalf];

emit AllPositionsRepaid(onBehalf, totalDebt);
```

	Impact As a result, an off chain indexer that reconstructs per position interest and penalty revenue from RepaidFixedLoanPosition events sees nothing for any position cleared through repayAll, only the grand total in AllPositionsRepaid.
Recommendation	Consider either emitting a RepaidFixedLoanPosition event per position inside repayAll, mirroring what repay emits so indexers receive the same per position breakdown regardless of which path cleared the position, or explicitly document that AllPositionsRepaid is the intended coarse grained substitute for repayAll and adjust any off chain consumers to treat it as such.
Status	Fixed, now emits an individual RepaidFixedLoanPosition event for all repaid positions for a successful repayAll call.

[I05] Unused imports and dead using directives across the CreditBroker contracts

Contract	CreditBrokerOperatorLib.sol
Severity	Informational
Description	Several import statements and using directives in the CreditBroker contract set are declared but never referenced. They add no functionality and reduce readability by suggesting dependencies that do not actually exist. The following instances were identified: 1. In CreditBroker.sol at line 17, Market and Position are imported from IMoolah.sol but never referenced. Only Id, IMoolah, and MarketParams are actually used from that import. This is pre-existing, though the PR already removed the SharesMathLib and UtilsLib imports in this file and was the natural place to also drop these. 2. In CreditBroker.sol at lines 16 and 40, the import { MarketParamsLib } statement and the using MarketParamsLib for MarketParams; directive are unused. No .id() call, or any MarketParamsLib.* call, remains. MARKET_ID is read from storage and _getMarketParams relies on MOOLAH.idToMarketParams. This is pre-existing. 3. In CreditBrokerOperatorLib.sol at line 13, Position is imported from IMoolah.sol but never referenced. Borrow shares are read as ctx.moolah.position(...).borrowShares without naming the type. This was introduced by this PR, since the file is new. 4. In CreditBrokerMath.sol at line 9, MarketParams is imported but never referenced. Market, Position, Id, and IMoolah are all used, but MarketParams is not. This is pre-existing.

5. In CreditBroker.sol at line 41, the using EnumerableSet for EnumerableSet.AddressSet; directive is unused because no AddressSet is declared in the contract. This is pre-existing and unrelated to the PR.

Impact

None of these carry a security impact. They are code hygiene issues that can mislead readers about the contract's real dependencies and add noise to the codebase.

Recommendation

Consider removing each unused symbol and directive.

Status

Fixed, removed the Position import this PR introduced in CreditBrokerOperatorLib. For the remaining four instances that are pre-existing and outside this PR's diff it is left untouched for future considerations and upgrades.

[I06] PR "isolation" wording is broader than Moolah's aggregate minLoan floor

Contract

CreditBroker.sol
CreditBrokerOperatorLib.sol
Moolah.sol

Severity

Informational

Description

Description

After this PR, CreditBroker validates only the fixed position touched by a call (`_validateFixedPosition / borrow-side amount >= minLoan`) instead of looping every position with `_validatePositions`. That correctly stops a single stranded broker position from reverting unrelated partial repay or compliant new borrow at the broker layer.

However, Moolah still enforces an account-level borrow floor in `_checkBorrowAssets`: after any repay, remaining borrow assets must be 0 or $\geq \text{minLoan}(\text{market})$. Example: `minLoan = 30`, positions 100 (healthy) + 20 (stranded). The broker allows repay on the healthy posId, but fully repaying 100 leaves Moolah borrow ≈ 20 , which reverts with remain borrow too low.

The new isolation tests cover partial repay and new borrow, not this full-repay-healthy case. Escape paths that succeed are (1) fully clear the stranded position first, then operate on the healthy one, or (2) `repayAll`.

Related Code

```
JavaScript
# PR #238 test plan / description
Isolation: a stranded position no longer blocks repay or borrow of
others,
while a new below-floor borrow is still rejected.
```

```

//CreditBrokerOperatorLib.sol
/// @dev A position must either be cleared or stay at or above Moolah's
minimum loan.
///     Only the position a call actually touched is checked:
validating the whole array
///     would let one stranded position block every other action on the
account.

//CreditBroker.t.sol
/// @notice A position stranded in the (0, minLoan) band must not block
repayment of an
///     unrelated healthy position. The old global
`_validatePositions` looped every
///     position and reverted "below min loan" here.

```

Impact

This is residual liveness / documentation precision, not fund theft and not a regression. Before the fix, full repay of the healthy leg was already blocked by global `_validatePositions`.

The PR still materially improves the reported brick: partial repay and new borrow no longer fail solely because another position is stranded.

Integrators or UI copy that state “a stranded position no longer blocks repay of others” without the Moolah aggregate caveat can mis-set user expectations and push failed full-repay txs.

Recommendation

Narrow PR / docs / frontend wording to: stranded positions no longer block partial repay of other positions or new borrows above `minLoan`. Fully exiting other debt may still require clearing the stranded leg or calling `repayAll` because of Moolah’s aggregate floor.

Add a Foundry test that expects remain borrow too low when fully repaying a healthy position while another position remains in `(0, minLoan)`.

Status

Fixed, narrowed to "partial repay and new borrows above `minLoan`" in `_validateFixedPosition`, in `repayAll`'s natspec, in the isolation test and in the PR description, each stating that Moolah's account-level floor still rejects a full repay that would leave the aggregate inside the band.

Additionally, ListaDao added the `test_fullRepayHealthy_whileOtherStranded_revertsAtMoolah` to represent the scenario

when expectations for borrows remain too low when fully repaying a healthy position while another position remains in (0, minLoan).

[I07] Enumerate live dead-band positions before upgrade go-live

Contract CreditBroker.sol
CreditBrokerOperatorLib.sol (ops / deployment)

Severity **Informational**

Description

Shipping this PR does not by itself clear accounts already stuck in the (0, minLoan) dead band on mainnet. The upgrade only changes validation and adds repayAll; existing FixedLoanPosition storage and Moolah borrow shares remain until users (or ops) act.

Without a pre-upgrade inventory, the team cannot know how many users / positions / notional are currently stranded under the live minLoan (and how that set would change if minLoanValue or loan-token price moves around the upgrade window).

Related Code

```
JavaScript
//CreditBrokerOperatorLib.sol
function _validateFixedPosition(OperatorContext memory ctx,
FixedLoanPosition memory position) internal view {
    if (position.isBadDebt) return;
    uint256 remaining = position.principal - position.principalRepaid;
    if (remaining == 0) return;
    require(remaining >=
ctx.moolah.minLoan(ctx.moolah.idToMarketParams(ctx.marketId)), "below
min loan");
}

//CreditBroker.sol
require(amount >= MOOLAH.minLoan(_getMarketParams(MARKET_ID)), "below
min loan");

// Moolah.sol
require(_checkBorrowAssets(marketParams, onBehalf),
ErrorsLib.REMAIN_BORROW_TOO_LOW);

// Moolah.sol
function _checkBorrowAssets(MarketParams memory marketParams, address
account) internal view returns (bool) {
    Id id = marketParams.id();
    if (position[id][account].borrowShares == 0) {
        return true;
    }
}
```

Description

```

return
    uint256(position[id][account].borrowShares).toAssetsDown(
        market[id].totalBorrowAssets,
        market[id].totalBorrowShares
    ) >= minLoan(marketParams);
}

```

Impact

This is operational readiness, not a new on-chain bug.

Blind go-live risks under-communicating to affected users, support load when partial repay still fails on the stranded leg, and no clear rollout plan for repayAll / frontend prompts. If minLoan rises again after upgrade, the stranded set can grow; inventory should be re-checked against the go-live floor.

Recommendation

Before timelock upgrade and UI cutover, snapshot CreditBroker markets: for each user position with remaining principal r where $0 < r < \text{minLoan}(\text{market})$ (and optionally Moolah aggregate borrow in that band), record user, posId, remaining principal, and whether other healthy positions exist on the same account.

Publish counts and notional; notify affected users that the stranded leg still needs full clear or repayAll; gate go-live on that checklist. Re-run the query at the upgrade block (and after any setMinLoanValue / oracle move).

Status

Acknowledged, ListaDao has indicated that this is non-applicable given the market has on 0x523809244d5ec5633a17fee837c16c69a0c6cf3b5596486a6c22b3000ecda5ad has no borrow positions. The stranded set is empty, so there is nothing to inventory or notify and go-live is not gated on it.

The above recommendation will be considered for a monitoring query once the market takes its first borrow, and is being re-run at the upgrade block and after any setMinLoanValue or oracle move.